

05

깃허브로 협업하기

05-1 서로 다른 컴퓨터에서 원격 저장소 함께 사용하기

05-2 원격 브랜치 정보 가져오기

05-3 협업의 기본 알아보기

05-4 원격 저장소에서 협업하기



원격 저장소 복제하기 — git clone - (1)

1. 04장에서 깃허브에 만든 'test-1'이라는 원격 저장소를 계속 사용
깃허브에 있는 test-1 저장소를 'git_home'이라는 지역 저장소로 복제
저장소 파일 목록 화면에서 [Code]를 클릭하면 복제할 주소를 확인할 수 있음
여기에서는 HTTPS 방식을 사용하기 위해 [HTTPS] 탭을 선택한 후  아이콘을 클릭하여 원격 저장소의 주소를 복사
2. 터미널 창에서 홈 디렉터리로 이동
다음처럼 입력하면 저장소를 복제하면서 동시에 git_home 디렉터리도 만들 수 있음
git_home이라는 디렉터리가 없다면 자동으로 디렉터리가 만들어짐

```
$ git clone 원격_저장소_주소 git_home
```

3. 원격 저장소의 내용을 git_office 디렉터리에도 복제
2번 단계에서 사용했던 소스 중에서 git_home 부분만 git_office로 수정하고, 나머지 소스는 똑같이 입력하면 됨
개인 컴퓨터와 회사 컴퓨터 양쪽에 복제됐다고 생각

```
$ git clone 원격_저장소_주소 git_office
```

원격 저장소 복제하기 — git clone - (2)

4. 터미널 창에서 ls 명령을 사용해서 git_home 디렉터리와 git_office 디렉터리가 만들어졌는지 확인

```
$ ls -la
```

5. git_home 저장소와 git_office 저장소에 똑같은 원격 저장소를 복제했기 때문에, git log 명령으로 확인해 보면 같은 커밋이 들어 있음
우선 git_home 디렉터리로 이동한 후 로그를 확인

```
$ cd git_home  
$ git log
```

6. 이제 git_office 디렉터리로 이동한 후 다시 로그를 확인해 보면 git_home과 똑같음

```
$ cd ~/git_office  
$ git log
```

원격 저장소 복제하기 — git clone - (3)

7. 이렇게 원격 저장소를 복제하면 지역 저장소와 원격 저장소가 자동으로 연결됨
git_home 디렉터리나 git_office 디렉터리에서 git remote -v 명령을 사용하면 원격 저장소에 연결된 것을 볼 수 있음

```
$ git remote -v
```

집에 있는 컴퓨터에서 작업하고 올리기 - (1)

1. git_home 디렉터리로 이동한 후 새로운 커밋을 만들

f1.txt 문서를 열고 간단한 내용을 추가

추가하는 내용이 중요한 것이 아니라 커밋하는 것이 중요한데 여기에서는 f1.txt 문서에 'c'를 추가

```
$ cd ~/git_home  
$ vim f1.txt
```

2. f1.txt 파일을 스테이징하고 커밋

git commit 명령에서 -am 옵션을 사용하면 스테이징과 커밋을 한꺼번에 할 수 있음

커밋 메시지는 'add c'라고 지정하고 git push 명령을 사용해 커밋을 원격 저장소에 올림

```
$ git commit -am "add c"  
$ git push
```

집에 있는 컴퓨터에서 작업하고 올리기 - (2)

3. 이제 깃허브의 원격 저장소로 접속해서 커밋이 제대로 올라왔는지 확인
파일 목록에서 f1.txt 오른쪽을 보면 방금 커밋한 메시지 add c가 보임
이렇게 원격 저장소의 내용을 복제한 지역 저장소에서 내용을 수정하고 커밋한 후 다시 원격 저장소에 올렸음

회사 컴퓨터에서 내려받아 작업하기 - (1)

1. 앞에서 원격 저장소를 복제했기 때문에 git_office 저장소는 이미 origin에 연결되어 있음
터미널 창에서 git_office 디렉터리로 이동한 후 git pull 명령을 사용해서 원격 저장소에 새로 올라온 커밋을 가져옴

```
$ cd ~/git_office  
$ git pull
```

2. 이번에는 git_office 디렉터리에서 f1.txt 파일을 수정
빔에서 f1.txt를 열면 집에 있는 컴퓨터(git_home)에서 수정했던 c가 포함되어 있음
회사 컴퓨터(git_office)에서는 영문자 'd'를 추가하고 저장

```
$ vim f1.txt
```

3. 이제 'add d'라는 메시지와 함께 커밋을 만들고, 원격 저장소로 푸시

```
$ git commit -am "add d"  
$ git push
```

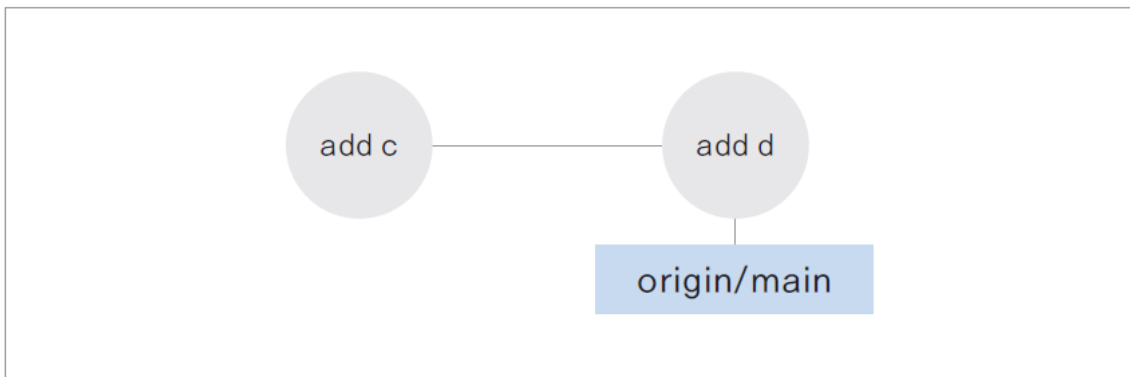
회사 컴퓨터에서 내려받아 작업하기 - (2)

4. 웹 브라우저에서 깃허브로 가서 저장소 파일 목록을 살펴보면 add d라는 커밋 메시지가 보임
커밋을 좀 더 자세하게 확인하고 싶다면, 파일 목록 위에 있는 [5 commits]를 클릭
5. 저장소에 add c와 add d 커밋이 차례대로 올라와 있는 것을 확인할 수 있음
6. 다시 집 컴퓨터(git_home 디렉터리)에서 작업할 때는 git pull 명령으로 원격 저장소에 있는 최신 커밋을 가져와 작업을 시작
git log 명령으로 확인해 보면 회사 컴퓨터(git_office 디렉터리)에서 푸시했던 'add d'라는 커밋이 개인 컴퓨터에도 들어와 있음

```
$ cd ~/git_home  
$ git pull  
$ git log
```


원격 main 브랜치 - (1)

1. 깃허브에서 test-1 저장소로 접속한 후 커밋 개수가 표시된 부분을 클릭
이 책의 실습을 따라왔다면 [5 commits]으로 되어 있을 것이고, 이 부분을 클릭하면 'add d'라는 커밋이 마지막임
이 상태에서 커밋 포인터가 어디를 가리키는지 생각해 보면
원격 저장소에 있는 HEAD는 원격 저장소의 main 브랜치를 가리키고, 원격 main 브랜치는 'add d'라는 최종 커밋을 가리킬 것임

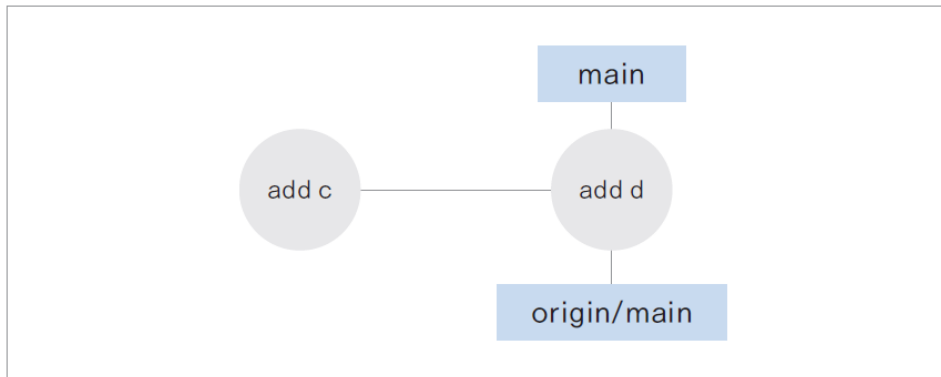


원격 main 브랜치 - (2)

2. 터미널 창에서 git_home 디렉터리로 이동한 후 git log 명령으로 커밋 상태를 확인
최종 커밋인 add d 앞에 (HEAD -> main, origin/main, origin/HEAD)라고 표시되어 있음

```
$ cd ~/git_home  
$ git log
```

(HEAD -> main)은 이 커밋이 지역 저장소의 최종 커밋이라는 뜻이고, (origin/main)는 원격 저장소의 최종 커밋이라는 뜻
아직 git_home 디렉터리가 원격 저장소를 복제한 상태 그대로이므로 지역 저장소와 원격 저장소의 최종 커밋이 같음



원격 main 브랜치 – (3)

3. git_home 디렉터리에 새로운 커밋을 만들기 위해 vim을 사용해 f3.txt 파일을 만든 후 간단히 'a'라고 입력
파일 내용은 어떤 것을 입력해도 상관없으며 여기에서는 새 파일을 만드는 것이 중요

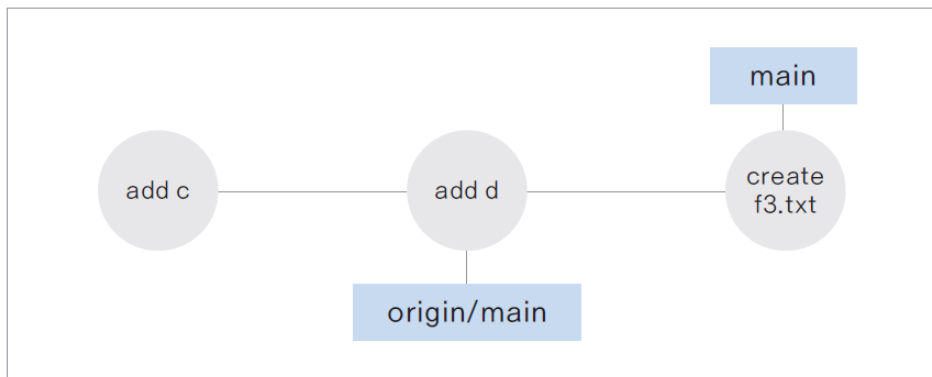
```
$ vim f3.txt
```

4. f3.txt를 스테이지에 올린 후 커밋을 만들고 커밋 메시지는 'create f3.txt'라고 함

```
$ git add f3.txt  
$ git commit -m "create f3.txt"
```

원격 main 브랜치 - (4)

5. `git log --oneline` 명령을 사용해 커밋 로그를 한눈에 확인
(HEAD -> main)는 방금 커밋한 create f3.txt를 가리킴
지역 저장소의 최종 커밋이 create f3.txt 커밋이라는 뜻
하지만 (origin/main, origin/HEAD)는 아직 add d 커밋을 가리키고 있음

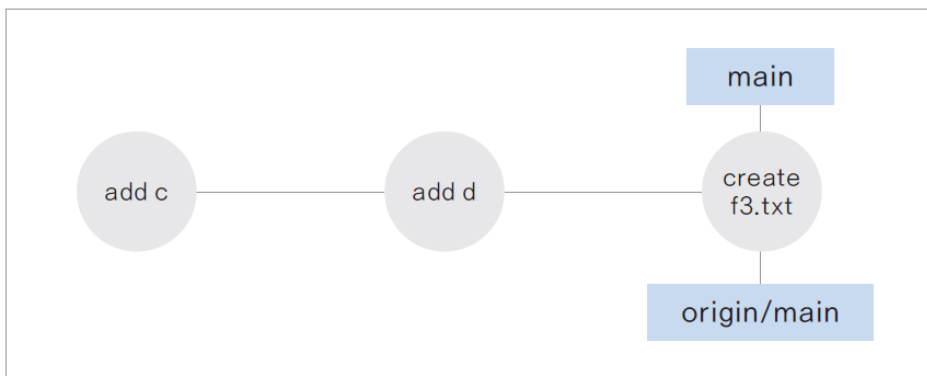


6. 이 상태에서 `git status`를 입력
현재 지역 저장소의 main 브랜치가 원격 main 브랜치의 버전보다 하나 앞서 있음
그리고 `git push` 명령으로 지역 저장소의 커밋을 원격 저장소로 올리라고 알려 줌

원격 main 브랜치 – (5)

7. 지역 저장소의 커밋을 원격 저장소로 올리기 위해 이제 git push 명령을 사용하고 그 후에 커밋 로그를 확인
푸시하기 전까지는 main 브랜치와 origin/main 브랜치가 가리키는 커밋이 달랐지만,
푸시한 후에는 main와 origin/main 브랜치가 같은 커밋을 가리킴

```
$ git push  
$ git log --oneline
```



원격 브랜치 정보 가져오기 — git fetch - (1)

1. 현재 깃허브 저장소에는 create f3.txt 커밋까지 올라간 상태

이번에는 회사 컴퓨터 저장소로 이동해서 진행

터미널 창에서 git_office 디렉터리로 이동한 후 git fetch 명령을 입력하면 원격 저장소에서 무언가 가져옴

```
$ cd ~/git_office  
$ git fetch
```

2. ls -al 명령을 사용해서 어떤 파일이 있는지 살펴봄

분명히 원격 저장소에 있던 커밋을 가져왔는데 git_home에서 원격 저장소로 푸시했던 f3.txt 파일이 보이지 않음

```
$ ls -al
```

원격 브랜치 정보 가져오기 — git fetch - (2)

3. git log 명령을 사용해서 커밋 로그를 살펴봄

커밋 해시 오른쪽에는 (HEAD -> main)만 보이고 원격 저장소의 origin/main은 보이지 않음
원격 저장소의 최신 커밋 정보를 가져왔지만 아직 지역 저장소에 합치지 않아
git_office에 있던 최신 커밋(HEAD -> main)만 나타나기 때문

```
$ git log --oneline
```

4. git status 명령으로 현재 깃의 상태를 확인

현재 브랜치가 origin/main에 비해 1개의 커밋이 뒤처져 있다고 나옴
즉, 원격 저장소의 최신 커밋 하나가 아직 지역 저장소에 반영되지 않았다는 뜻
git pull 명령을 사용하면 지역 저장소를 업데이트할 수 있다고 알려 줌

원격 브랜치 정보 가져오기 — git fetch - (3)

5. 페치로 가져온 커밋 정보를 확인하려면 git diff 명령을 사용해서
현재 최신 커밋과 원격 저장소에서 가져온 커밋의 차이를 살펴보면 됨
f3.txt 파일이 새로 생겼음

```
$ git diff HEAD origin/main
```

6. 원격 저장소의 커밋을 확인하고 지역 저장소에 합치겠다고 결정했다면 merge 명령을 사용해 합치면 됨

```
$ git merge origin/main
```

7. 다시 깃 로그를 살펴보면 'create f3.txt'라는 최신 커밋이 지역 저장소에 반영된 것을 볼 수 있음

```
$ git log --oneline
```


협업을 위한 저장소 만들기

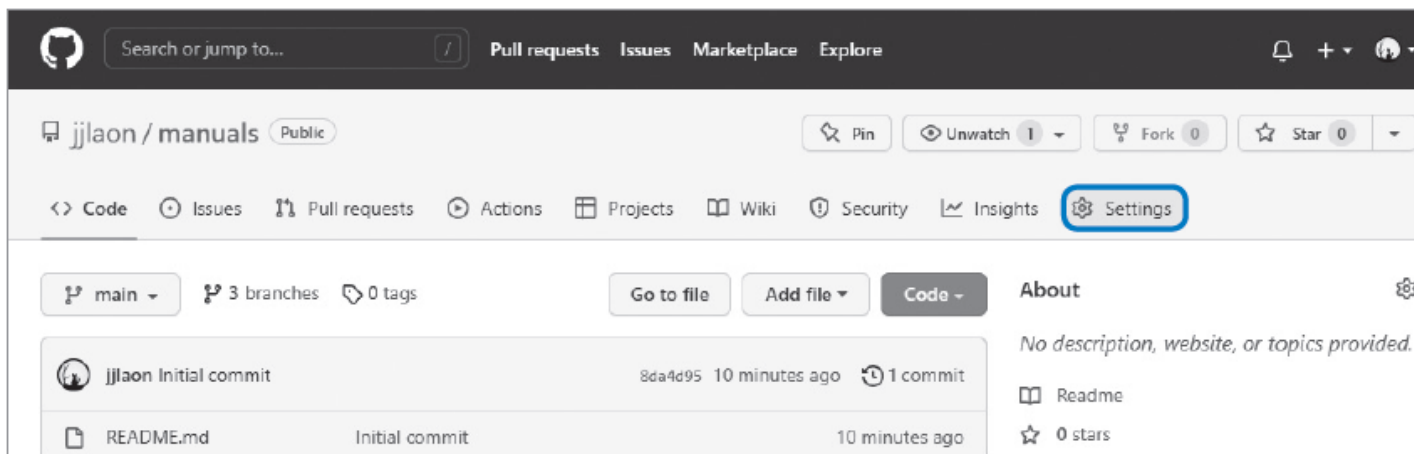
- 깃허브의 공개 저장소는 주소만 알면 누구든지 접속해서 모든 소스를 살펴볼 수 있음
- 깃허브 회원이라면 누구나 오픈 소스 프로젝트의 소스를 내려받을 수도 있음
하지만 누구나 저장소에 커밋을 푸시할 수는 없음
- 공개 저장소든 비공개 저장소든 승인된 공동 작업자에게만 커밋을 올릴 수 있는 권한을 주어야 함
그래야 프로젝트 소스가 무분별하게 수정되는 일을 막을 수 있음
- 여기에서는 팀장과 팀원 1, 팀원 2 이렇게 세 사람이 깃허브를 통해 협업하고,
팀장이 저장소를 만든 후 팀원 1과 팀원 2를 공동 작업자로 추가한다고 가정
팀장이 'manuals'라는 깃허브 저장소를 만들 때 'Add a README file'에 체크한 후
[Create repository]를 클릭하면 main 브랜치가 만들어짐

저장소에 브랜치 만들기

1. 깃허브 저장소에서 [main]이라고 브랜치가 표시된 부분을 클릭하면 브랜치를 추가하거나 전환할 수 있음
새로 추가할 apple을 입력한 후 그 아래 표시된 [Create branch apple from 'main']을 클릭
2. apple 브랜치가 추가되면서 apple 브랜치로 전환됨
같은 방법으로 ms 브랜치도 추가할 수 있음
이제 브랜치 목록을 클릭하면 기본 main 브랜치 외에 apple 브랜치와 ms 브랜치가 생겨 있고,
사용자마다 이 브랜치들로 자신의 커밋을 푸시할 것

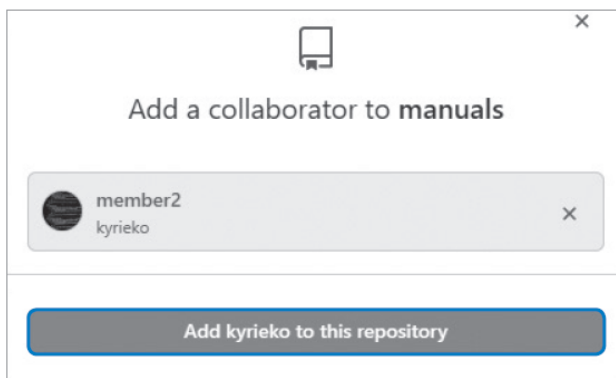
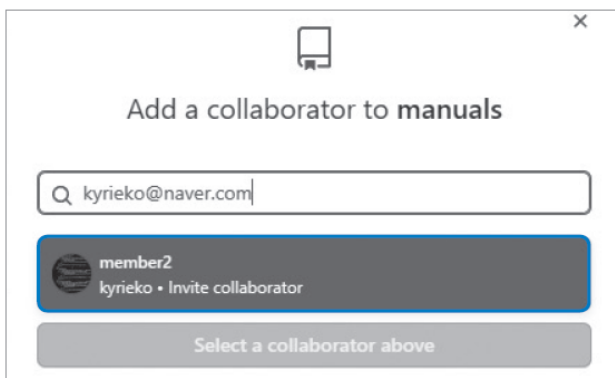
공동 작업자 추가하기 - (1)

1. 원격 저장소를 만들고 브랜치까지 만들었다면 이제 같이 작업할 공동 작업자를 추가하면 됨
원격 저장소를 관리하는 사람(예를 들어 팀장)이 manual 저장소에서 [Settings]를 클릭



공동 작업자 추가하기 - (2)

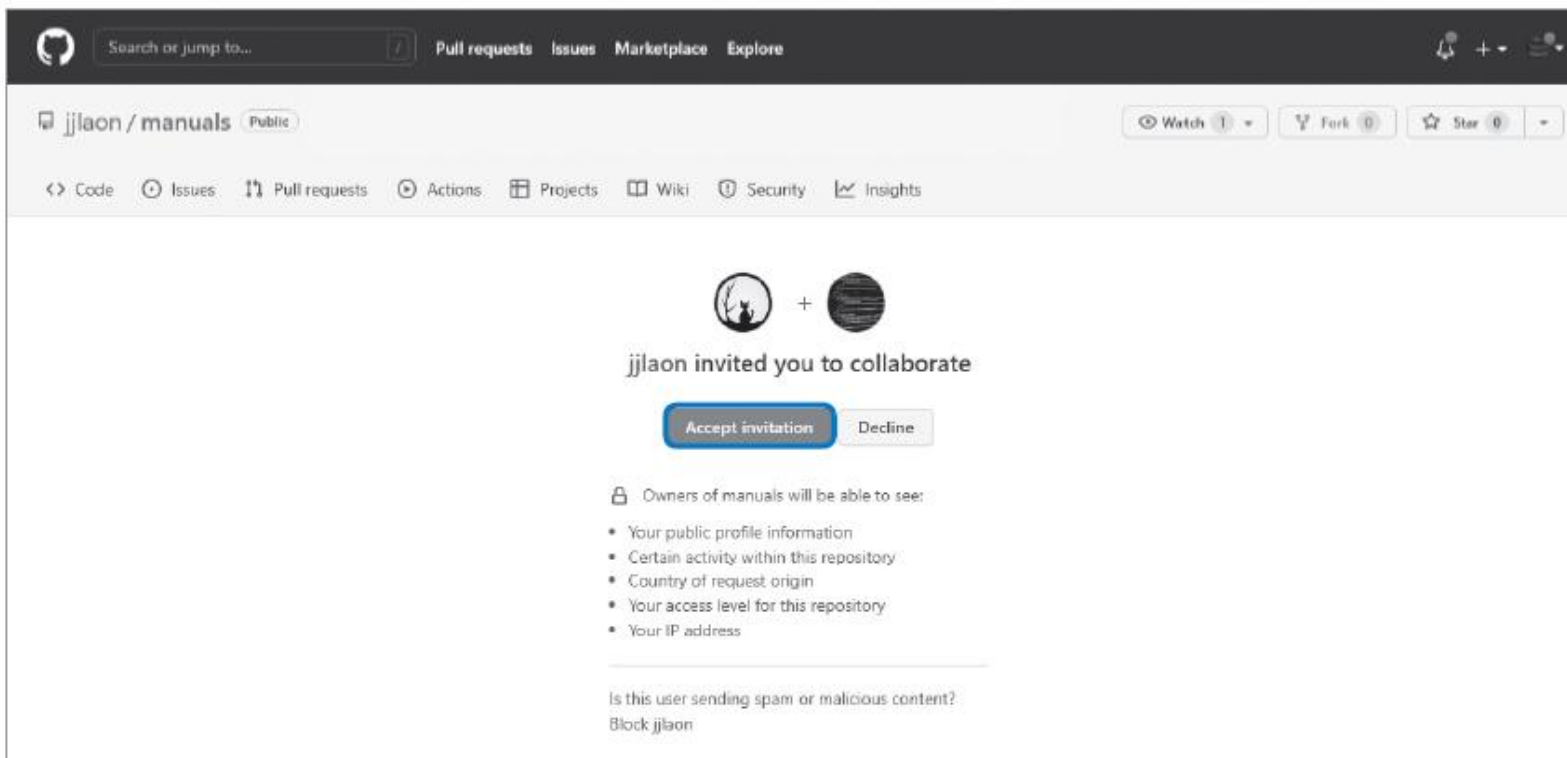
2. 왼쪽 항목 중 [Collaborators]를 선택한 후 [Add People]을 클릭하면
원격 저장소를 같이 사용하고 커밋할 수 있는 작업자를 추가할 수 있음
3. 공동 작업자의 이메일 주소를 입력하다 보면 입력 창 아래에 완성된 이메일 주소가 나타남
완성된 이메일 주소를 클릭한 후 [Add 메일 주소 또는 사용자 이름 to this repository]를 클릭



4. 같은 방법으로 필요한 공동 작업자를 계속 추가할 수 있음

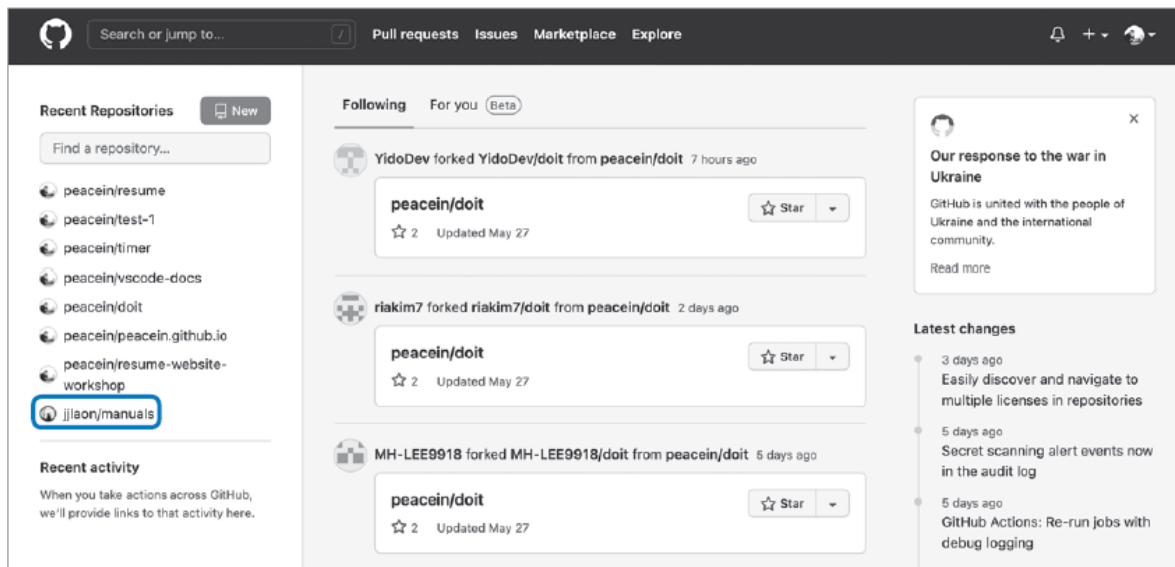
공동 작업자 추가하기 - (3)

5. 저장소에 초대받은 사용자는 협업자로 초대됐다는 이메일과 깃허브 메시지를 받게 되는데, 깃허브 메시지 화면에서 [Accept Invitation]을 클릭하면 초대를 수락하는 것이 됨



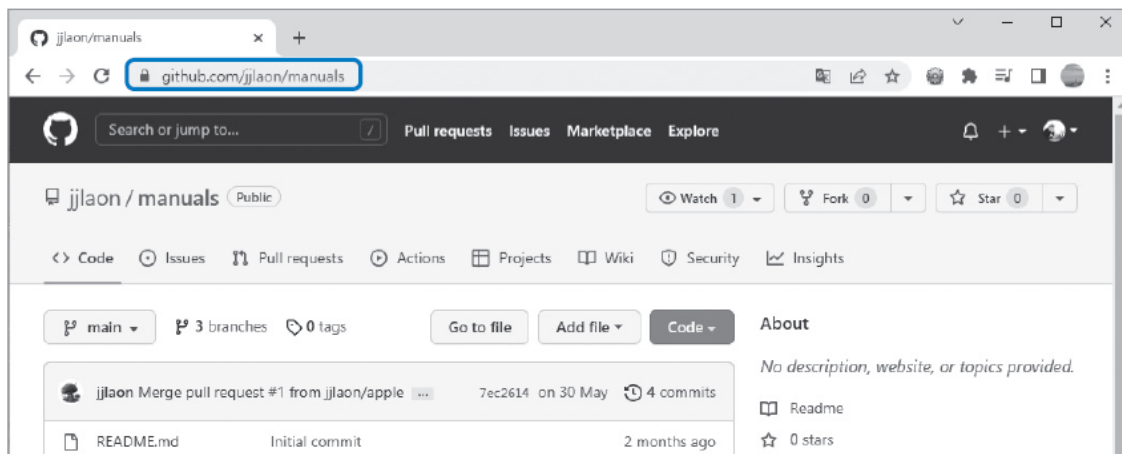
원격 저장소 복제하기 - (1)

1. 공동 작업자가 깃허브에 접속하면 저장소 목록에 협업 저장소도 함께 나타남
예를 들어 'peacein'라는 사용자를 공동 작업자로 추가했다고 가정
peacein이라는 사람이 깃허브에 접속해서 자신의 저장소 목록을 살펴보면 자신이 협업하고 있는 저장소, 즉 jjlaon이 만든 manuals 저장소도 목록에 나타남
저장소 목록에서 저장소 이름 앞에 자신의 아이디가 붙은 것은 자기가 직접 만든 저장소이고, 다른 사람의 아이디가 붙은 저장소는 협업 저장소라고 생각하면 됨
협업 저장소를 클릭!



원격 저장소 복제하기 - (2)

2. 맨 위에 협업 저장소의 주소가 나타나는데 이 주소를 복사해 두고,
이 주소 뒤에 .git을 붙이면 저장소 주소가 됨



3. 터미널에서 git clone 명령 뒤에 원격 저장소 주소를 붙여 넣음
그리고 뒤에 지역 저장소 디렉터리 이름까지 적으면 그 디렉터리로 저장소가 복제

```
$ git clone 원격_저장소_주소 디렉터리명
```

원격 저장소 복제하기 - (3)

4. 지역 저장소 디렉터리로 이동한 후 `git log` 명령으로 어떤 커밋이 있는지 확인
원격 저장소를 만들면서 함께 추가한 README 파일의 커밋만 보임
그리고 이 커밋은 지역 저장소의 최신 커밋일 뿐만 아니라
원격 저장소의 main과 apple, ms 브랜치의 최신 커밋이라는 걸 알 수 있음

```
$ cd manuals  
$ git log
```

5. 원격 저장소를 복제했다면 앞으로 자신이 직접 커밋을 만들고 깃허브로 푸시할 수 있음
그런데 그 전에 커밋할 때 사용할 이름과 이메일 주소를 설정해야 함
아직 지역 저장소 디렉터리에 있지 않다면 지역 저장소 디렉터리로 이동한 후 `git config` 명령을 사용해 이름과 주소를 지정

```
$ cd manuals  
$ git config user.name "사용자 이름"  
$ git config user.email "메일 주소"
```


원격 저장소에 커밋 푸시하기 - (1)

1. 공동 작업을 할 때는 내가 커밋을 올리기 전에 누군가가 원격 저장소에 커밋을 푸시했을 수도 있으므로 항상 풀을 먼저 해야 함
변경 사항이 있다면 내려받아서 지역 저장소에 병합해 주고, 변경 사항이 없다면 'Already up to date'라고 표시됨

```
$ git pull
```

2. apple 브랜치로 전환한 후 vim을 사용해 apple.txt 문서를 작성하는데 문서의 내용은 어떤 것이든 상관없음
apple.txt 문서를 스테이지에 올리고 '초안작성'라는 메시지와 함께 커밋

```
$ git switch apple  
$ vim apple.txt  ← apple.txt에 내용 입력 후 저장  
$ git add apple.txt  
$ git commit -m "초안작성 "
```

원격 저장소에 커밋 푸시하기 - (2)

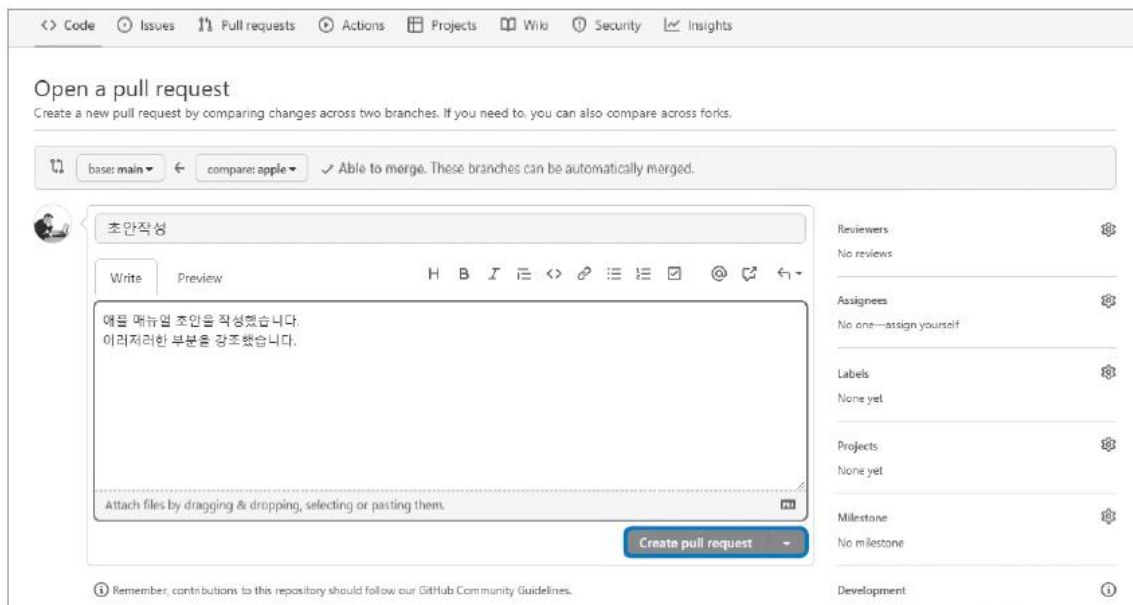
3. git push 명령을 사용해 방금 만든 커밋을 깃허브에 올림
처음 푸시할 때는 -u 옵션을 붙이고, 깃허브의 apple 브랜치로 푸시할 것이기 때문에 'origin apple'처럼 브랜치 이름을 넣어 줌
푸시가 끝나면 프롬프트(\$)가 나타남

```
$ git push -u origin apple
```

4. 깃허브에 제대로 올라갔는지 확인하려면 manuals 저장소에서 main이라고 된 브랜치 목록을 클릭한 후 [apple]을 선택해서 브랜치를 전환
방금 푸시한 '초안작성' 커밋이 apple 브랜치에 올라온 것을 확인할 수 있음

풀 리퀘스트 보내기 및 병합하기 - (1)

1. 앞의 실습에서 apple 브랜치에 '초안작성'이라는 커밋을 푸시했음
apple 브랜치에 푸시했기 때문에 main 브랜치에는 커밋이 병합되지 않은 상태
파일 목록 위에 있는 [Contribute]를 클릭한 후 [Open pull request]를 클릭
2. 다음과 같이 풀 리퀘스트 메시지를 작성한 후 [Create pull request]를 클릭하면 협업 중인 저장소에 풀 리퀘스트가 전송



Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also compare across forks.

base: main ← compare: apple ✓ Able to merge. These branches can be automatically merged.

초안작성

Write Preview H B I

예를 매뉴얼 초안을 작성했습니다.
이리저리한 부분을 강조했습니다.

Attach files by dragging & dropping, selecting or pasting them.

Create pull request

Remember, contributions to this repository should follow our GitHub Community Guidelines.

Reviewers
No reviews

Assignees
No one—assign yourself

Labels
None yet

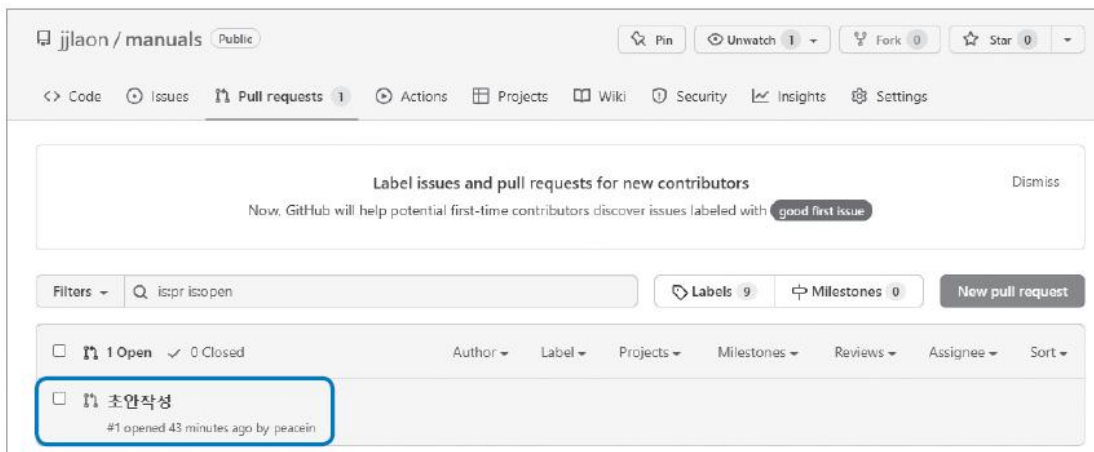
Projects
None yet

Milestone
No milestone

Development

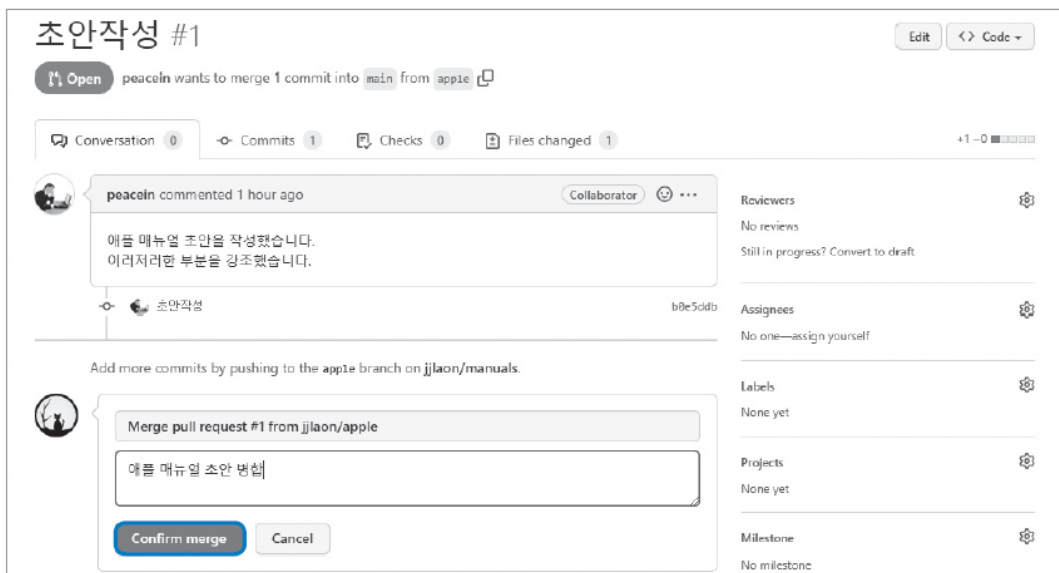
풀 리퀘스트 보내기 및 병합하기 - (2)

3. 협업 중인 원격 저장소에 등록된 풀 리퀘스트는 공동 작업자 누구나 살펴보고 병합할 수 있음
공동 작업자 가운데 누군가 풀 리퀘스트를 요청하면 저장소 파일 목록 위의 [Pull request]에 숫자가 표시됨
[Pull Request]를 클릭
4. 공동 작업자들이 등록한 풀 리퀘스트가 목록 형태로 나타남
아직 마무리되지 않은 풀리퀘스트는 Open 상태라고 하는데, 풀 리퀘스트 제목 앞에 가 붙은 것은 Open 상태의 풀리퀘스트임
살펴볼 풀 리퀘스트를 선택



풀 리퀘스트 보내기 및 병합하기 - (3)

5. 풀 리퀘스트 메시지를 살펴본 뒤 내용에 문제가 없으면 [Merge pull request]를 클릭해 병합
필요하다면 이 공간을 통해 풀 리퀘스트를 남긴 사람과 메시지를 주고받을 수도 있음
6. 커밋을 브랜치로 병합하는 것도 하나의 커밋이므로, 커밋 메시지를 입력해야 함
기본 메시지를 사용하거나 원하는 내용을 입력할 수 있음
입력한 후에는 [Confirm merge]를 클릭



The screenshot shows a GitHub pull request interface for a repository named 'apple'. The pull request is titled '초안작성 #1' (Draft #1) and is from the 'apple' branch to the 'main' branch. The user 'peacein' has commented: '애플 매뉴얼 초안을 작성했습니다. 이리저리한 부분을 강조했습니다.' (I have written the Apple manual draft. I have highlighted the parts that need attention.)

The interface includes a 'Merge pull request' button and a 'Confirm merge' button. The 'Confirm merge' button is highlighted in blue. The interface also shows a list of reviewers, assignees, labels, projects, and milestones, all of which are currently empty.

풀 리퀘스트 보내기 및 병합하기 - (4)

7. 풀 리퀘스트가 성공적으로 병합되었다는 메시지가 나타나면 병합이 끝난 것
그리고 Open 상태였던 풀 리퀘스트가 Merged 상태로 바뀜
8. 화면 왼쪽 위에 있는 [Code]를 클릭해서 파일 목록으로 돌아가 보면,
apple 브랜치로 푸시했던 '초안작성' 커밋이 main 브랜치로 병합된 것을 볼 수 있음